小磨推智能营销系统技术文档

程序鉴别材料

文档编号: XMT-TECH-2024-001
版本号: V1.0
编写日期: 2024年12月
编写单位: 小磨推科技有限公司
密级: 内部公开

版权声明

© 2024 小磨推科技有限公司 版权所有。本文档包含的技术信息和商业机密，未经本公司书面许可，任何单位或个人不得以任何形式复制、传播或使用。

目录

第1章 封面信息

- 1.1 文档基本信息
- 1.2 版本历史
- 1.3 版权声明

第2章 目录

第3章 项目概述

- 3.1 项目背景

- 3.2 项目目标
- 3.3 系统概述
- 3.4 技术栈概览
- 3.5 核心功能模块
- 3.6 系统特色

第4章 架构设计

- 4.1 总体架构
- 4.2 技术架构
- 4.3 系统架构图
- 4.4 数据流架构
- 4.5 部署架构
- 4.6 安全架构

第5章 核心功能模块

- 5.1 用户认证模块
- 5.2 内容管理模块
- 5.3 设备管理模块
- 5.4 营销推广模块
- 5.5 数据分析模块
- 5.6 商户管理模块

第6章 技术实现细节

- 6.1 后端架构实现
- 6.2 数据库设计
- 6.3 API接口设计
- 6.4 安全机制实现
- 6.5 缓存策略
- 6.6 消息队列

第7章 性能优化

- 7.1 数据库优化

- 7.2 缓存优化
- 7.3 接口性能优化
- 7.4 前端性能优化

第8章 安全机制

- 8.1 身份认证
- 8.2 权限控制
- 8.3 数据加密
- 8.4 安全审计

第9章 部署与运维

- 9.1 环境要求
- 9.2 部署流程
- 9.3 监控告警
- 9.4 备份恢复

第1章 项目概述

1.1 项目背景

小磨推智能营销系统是一个基于微信小程序生态的智能营销推广平台，旨在为商户提供全方位的数字化营销解决方案。系统整合了内容创作、多平台发布、数据分析、用户行为追踪等核心功能，帮助商户提升营销效率和转化率。

1.2 项目目标

- **提升营销效率**：通过AI辅助内容创作和自动化发布流程，显著提升商户营销效率
- **数据驱动决策**：提供实时数据分析和用户行为洞察，支持商户做出精准营销决策
- **多平台整合**：支持微信、抖音、快手等多个主流社交平台的统一管理
- **智能化运营**：基于大数据和机器学习技术，提供智能推荐和个性化营销方案

1.3 系统概述

本系统采用前后端分离架构，后端基于ThinkPHP 8.0框架构建RESTful API服务，前端包含微信小程序客户端和Web管理后台。系统支持高并发访问，具备良好的扩展性和可维护性。

1.4 技术栈概览

后端技术栈

- **框架**: ThinkPHP 8.0
- **PHP版本**: PHP 8.1+
- **数据库**: MySQL 8.0+ / Redis 6.0+
- **缓存**: Redis
- **消息队列**: Think Queue
- **认证**: JWT (JSON Web Token)
- **文件存储**: 支持本地存储、七牛云、阿里云OSS

前端技术栈

- **小程序**: 微信小程序原生开发
- **管理后台**: Vue.js 3 + Element Plus
- **构建工具**: Vite
- **状态管理**: Vuex/Pinia

开发工具

- **版本控制**: Git
- **API文档**: Swagger/OpenAPI
- **测试框架**: PHPUnit
- **CI/CD**: Jenkins/GitHub Actions

1.5 核心功能模块

1. **用户认证模块**
- 微信小程序授权登录
- 手机号验证码登录
- JWT令牌管理
- 权限控制

2. **内容管理模块**
- AI内容生成
- 内容审核
- 多媒体素材管理
- 内容模板管理

3. **设备管理模块**
- NFC设备管理
- WiFi探针设备
- 设备状态监控
- 触发规则配置

4. **营销推广模块**
- 多平台内容发布
- 定时发布
- 推荐系统
- 营销活动管理

5. **数据分析模块**
- 用户行为分析
- 营销效果分析
- 实时数据展示
- 报表生成

6. **商户管理模块**
- 商户信息管理
- 联系方式配置
- 服务记录管理
- 统计数据

1.6 系统特色

- **AI赋能**: 集成百度文心一言、微信等AI服务，提供智能内容创作能力
- **多端同步**: 支持小程序、Web后台、APP多端数据同步
- **实时监控**: 设备状态实时监控，异常告警及时通知
- **灵活配置**: 支持自定义触发规则、推荐策略等灵活配置
- **高可用性**: 采用集群部署、负载均衡、故障转移等技术保障系统稳定性

第2章 架构设计

2.1 总体架构

小磨推智能营销系统采用分层架构设计，从上到下分为：

1. **表现层（Presentation Layer）**
- 微信小程序客户端
- Web管理后台
- 第三方接口调用

2. **应用层（Application Layer）**
- API网关
- 控制器（Controller）
- 中间件（Middleware）

3. **业务层（Business Layer）**
- 服务层（Service）
- 业务逻辑处理
- 工作流引擎

4. **数据访问层（Data Access Layer）**
- 数据模型（Model）
- 数据访问对象（DAO）
- ORM映射

5. **基础设施层（Infrastructure Layer）**
- 数据库
- 缓存
- 消息队列
- 文件存储

2.2 技术架构

```
┌──────────────────────────────────────────────────────────────┐
│                          客户端层                            │
├──────────────────────┬──────────────────────┬──────────────────┤
│   微信小程序         │    Web管理后台       │  第三方系统集成  │
│   - 内容创作         │    - 数据分析        │  - 平台API对接   │
│   - 数据查看         │    - 商户管理        │  - 数据同步      │
│   - 设备监控         │    - 系统配置        │                  │
└──────────────────────┴──────────────────────┴──────────────────┘
                              │
┌──────────────────────────────────────────────────────────────┐
│                         API网关层                            │
│  - 路由分发   - 认证授权   - 限流控制   - 日志记录            │
└──────────────────────────────────────────────────────────────┘
                              │
┌──────────────────────────────────────────────────────────────┐
│                         应用服务层                           │
├──────────────┬──────────────┬──────────────┬──────────────────┤
│ 用户服务     │ 内容服务     │ 设备服务     │ 营销服务         │
│ - 认证管理   │ - AI创作     │ - NFC管理    │ - 多平台发布     │
│ - 权限控制   │ - 审核流程   │ - WiFi配置   │ - 推荐算法       │
└──────────────┴──────────────┴──────────────┴──────────────────┘

┌──────────────┬──────────────┬──────────────┬──────────────────┐
│ 数据服务     │ 通知服务     │ 文件服务     │ 分析服务         │
│ - 数据统计   │ - 短信通知   │ - 上传下载   │ - 行为分析       │
│ - 报表生成   │ - 微信通知   │ - 云存储     │ - 效果评估       │
└──────────────┴──────────────┴──────────────┴──────────────────┘
                              │
┌──────────────────────────────────────────────────────────────┐
│                         数据存储层                           │
├──────────────┬──────────────┬──────────────┬──────────────────┤
│ MySQL        │ Redis        │ 消息队列     │ 文件存储         │
│ - 业务数据   │ - 缓存       │ - 异步任务   │ - 素材管理       │
│ - 用户信息   │ - 会话       │ - 延迟任务   │ - 备份归档       │
└──────────────┴──────────────┴──────────────┴──────────────────┘
```

2.3 系统架构图

2.3.1 整体系统架构

```
graph TB
    subgraph "客户端层"
        A[微信小程序]
        B[Web管理后台]
        C[第三方系统]
    end

    subgraph "网关层"
        D[Nginx负载均衡]
        E[API网关]
    end

    subgraph "应用层"
        F[认证模块]
        G[内容模块]
        H[设备模块]
        I[营销模块]
        J[数据模块]
    end

    subgraph "服务层"
        K[AI服务]
        L[推送服务]
        M[文件服务]
        N[分析服务]
    end

    subgraph "数据层"
        O[(MySQL主库)]
        P[(MySQL从库)]
        Q[(Redis集群)]
        R[消息队列]
        S[文件存储]
    end

    A --> D
    B --> D
    C --> D
    D --> E
    E --> F
    E --> G
    E --> H
```

```
    E --> I
    E --> J

    F --> K
    G --> K
    G --> M
    H --> L
    I --> L
    I --> N
    J --> N

    F --> O
    F --> Q
    G --> O
    G --> P
    G --> Q
    H --> O
    I --> O
    J --> P
    J --> Q

    R --> O
    S --> M
```

2.4 数据流架构

2.4.1 数据流向图

```
flowchart LR
    subgraph "数据采集"
        A1[用户行为]
        A2[设备数据]
        A3[业务数据]
    end

    subgraph "数据处理"
        B1[实时处理]
        B2[批量处理]
        B3[AI分析]
    end

    subgraph "数据存储"
        C1[业务数据库]
        C2[缓存数据库]
        C3[数据仓库]
    end

    subgraph "数据应用"
        D1[实时展示]
        D2[统计分析]
        D3[智能推荐]
    end

    A1 --> B1
    A2 --> B1
    A3 --> B2

    B1 --> C2
    B1 --> C1
    B2 --> C1
    B2 --> C3
    B3 --> C3

    C1 --> D2
    C2 --> D1
    C3 --> D2
    C3 --> D3
```

2.5 部署架构

2.5.1 生产环境部署架构

```
Internet
    │
    ▼
┌─────────────────────────────────────────┐
│                CDN/云分发                 │
│              静态资源加速                 │
└─────────────────────────────────────────┘
    │
    ▼
┌─────────────────────────────────────────┐
│                负载均衡层                 │
│              Nginx/HAProxy               │
└─────────────────────────────────────────┘
    │
    ▼
┌──────────────────┬──────────────────────┐
│  Web服务器集群   │      应用服务器集群   │
│  - 静态资源服务  │      - API服务        │
│  - 反向代理      │      - 业务逻辑       │
└──────────────────┴──────────────────────┘
    │                    │
    ▼                    ▼
┌─────────────────────────────────────────┐
│                数据服务层                 │
├──────────────┬──────────────┬────────────┤
│ MySQL主从    │  Redis集群   │  消息队列  │
│ 读写分离     │  多级缓存    │  异步处理  │
└──────────────┴──────────────┴────────────┘
    │
    ▼
┌─────────────────────────────────────────┐
│               监控运维层                 │
│  - 日志收集  - 性能监控  - 告警通知      │
└─────────────────────────────────────────┘
```

2.6 安全架构

2.6.1 安全体系架构

1. **网络安全**
- HTTPS/TLS加密传输
- 防火墙配置
- DDoS防护
- 安全组策略

2. **应用安全**
- JWT令牌认证
- API接口鉴权
- 参数验证与过滤
- SQL注入防护
- XSS攻击防护

3. **数据安全**
- 敏感数据加密存储
- 数据脱敏处理
- 数据备份与恢复
- 访问日志审计

4. **运维安全**
- 操作审计
- 权限最小化原则
- 定期安全扫描
- 漏洞管理

第3章 核心功能模块

3.1 用户认证模块

3.1.1 功能概述

用户认证模块是系统的核心安全模块，负责用户身份验证、权限管理和会话控制。支持多种登录方式，包括微信小程序授权、手机号验证码登录等。

3.1.2 核心功能

1. **多方式登录**
- 微信小程序授权登录

- 手机号验证码登录
- 管理员账号密码登录

2. **令牌管理**
- JWT令牌生成
- 令牌刷新机制
- 令牌黑名单管理

3. **权限控制**
- RBAC权限模型
- 接口级权限验证
- 数据级权限控制

3.1.3 技术实现

认证控制器实现:

```php
namespace app\controller;

use app\service\AuthService;
use think\Request;

class Auth extends BaseController
{
    protected AuthService $authService;

    /**
     * 微信小程序登录
     * @param Request $request
     * @return \think\Response
     */
    public function login(Request $request)
    {
        $data = $request->post();

        try {
            // 管理员登录
            if (!empty($data['username']) && !empty($data['password'])) {
                $this->validate($data, 'AdminAuth.login');
                $result = $this->authService->adminLogin(
                    $data['username'],
                    $data['password']
                );
                return $this->success($result, '登录成功');
            }

            // 微信小程序登录
            $scene = (!empty($data['encrypted_data']) && !empty($data['iv']))
                ? 'loginWithUserInfo' : 'login';
            $this->validate($data, 'WechatAuth.' . $scene);

            $result = $this->authService->wechatLogin(
                $data['code'],
                $data['encrypted_data'] ?? '',
                $data['iv'] ?? ''
            );

            return $this->success($result, '登录成功');
        } catch (\Exception $e) {
            return $this->error($e->getMessage(), 400, 'login_failed');
```

```
        }
    }

    /**
     * 刷新令牌
     * @param Request $request
     * @return \think\Response
     */
    public function refresh(Request $request)
    {
        $data = $request->post();

        try {
            $this->validate($data, 'WechatAuth.refresh');
            $result = $this->authService->refreshToken($data['refresh_token']);
            return $this->success($result, '刷新成功');
        } catch (\Exception $e) {
            return $this->error($e->getMessage(), 401, 'token_refresh_failed');
        }
    }
}
```

JWT中间件实现:

```
namespace app\middleware;

use app\service\AuthService;
use think\Request;
use think\Response;

class JwtAuth
{
    public function handle(Request $request, \Closure $next)
    {
        $token = $request->header('Authorization');

        if (!$token) {
            return json(['code' => 401, 'msg' => '未提供认证令牌'], 401);
        }

        // 移除 Bearer 前缀
        $token = str_replace('Bearer ', '', $token);

        try {
            $authService = new AuthService();
            $payload = $authService->verifyToken($token);

            // 将用户信息注入到请求中
            $request->user_id = $payload['user_id'];
            $request->user_type = $payload['user_type'];

            return $next($request);
        } catch (\Exception $e) {
            return json(['code' => 401, 'msg' => '令牌无效或已过期'], 401);
        }
    }
}
```

3.2 内容管理模块

3.2.1 功能概述

内容管理模块是系统的核心业务模块，提供AI辅助内容创作、内容审核、素材管理、模板管理等功能，支持多类型内容的生成和管理。

3.2.2 核心功能

1. **AI内容生成**
- 集成百度文心一言
- 多种内容类型支持
- 自定义生成参数

2. **内容审核**
- 敏感词过滤
- AI智能审核
- 人工审核流程

3. **素材管理**
- 多媒体素材上传
- 分类管理
- 标签系统

4. **模板管理**
- 内容模板创建
- 模板应用
- 模板共享

3.2.3 技术实现

AI内容服务实现:

```php
namespace app\service;

use GuzzleHttp\Client;
use think\facade\Cache;

class WenxinService
{
    private string $apiKey;
    private string $secretKey;
    private string $accessToken;
    private Client $client;

    public function __construct()
    {
        $this->apiKey = config('wenxin.api_key');
        $this->secretKey = config('wenxin.secret_key');
        $this->client = new Client([
            'timeout' => 30,
            'verify' => false
        ]);
        $this->initAccessToken();
    }

    /**
     * 生成营销内容
     * @param array $params
     * @return array
     */
    public function generateContent(array $params): array
    {
        $prompt = $this->buildPrompt($params);

        $data = [
            'messages' => [
                [
                    'role' => 'user',
                    'content' => $prompt
                ]
            ],
            'temperature' => $params['temperature'] ?? 0.8,
            'top_p' => $params['top_p'] ?? 0.9,
            'penalty_score' => $params['penalty_score'] ?? 1.0
        ];
```

```
        $response = $this->client->post(
            $this->getApiUrl('chat/completions'),
            [
                'json' => $data,
                'headers' => [
                    'Content-Type' => 'application/json',
                    'Authorization' => 'Bearer ' . $this->accessToken
                ]
            ]
        );

        $result = json_decode($response->getBody()->getContents(), true);

        if (!isset($result['result'])) {
            throw new \Exception('AI生成失败: ' . ($result['error_msg'] ?? '未知错误'));
        }

        return [
            'content' => $result['result'],
            'usage' => $result['usage'] ?? [],
            'request_id' => $result['id'] ?? ''
        ];
    }

    /**
     * 构建提示词
     * @param array $params
     * @return string
     */
    private function buildPrompt(array $params): string
    {
        $template = $params['template'] ?? '';
        $industry = $params['industry'] ?? '';
        $product = $params['product'] ?? '';
        $style = $params['style'] ?? '专业';
        $length = $params['length'] ?? '中等';

        $prompt = "请为以下产品生成营销推广内容: \n";
        $prompt .= "行业: {$industry}\n";
        $prompt .= "产品: {$product}\n";
        $prompt .= "风格: {$style}\n";
        $prompt .= "长度: {$length}\n";
```

```
        if ($template) {
            $prompt .= "模板参考：\n{$template}\n";
        }

        $prompt .= "\n要求：\n";
        $prompt .= "1. 内容要有吸引力，能够引起用户兴趣\n";
        $prompt .= "2. 突出产品特点和优势\n";
        $prompt .= "3. 包含明确的行动召唤\n";
        $prompt .= "4. 符合平台规范，不包含敏感词汇\n";

        return $prompt;
    }
}
```

内容控制器实现:

```
namespace app\controller;

use app\service\ContentService;
use app\service\WenxinService;
use think\Request;

class Content extends BaseController
{
    protected ContentService $contentService;
    protected WenxinService $wenxinService;

    /**
     * AI生成内容
     * @param Request $request
     * @return \think\Response
     */
    public function generate(Request $request)
    {
        $data = $request->post();

        try {
            $this->validate($data, [
                'type' => 'require|in:text,image',
                'prompt' => 'require|max:1000',
                'params' => 'array'
            ]);

            $result = $this->wenxinService->generateContent($data);

            // 保存生成的内容
            $contentId = $this->contentService->saveGeneratedContent(
                $this->request->user_id,
                $result,
                $data
            );

            return $this->success([
                'content_id' => $contentId,
                'content' => $result['content'],
                'usage' => $result['usage']
            ], '生成成功');

        } catch (\Exception $e) {
            return $this->error($e->getMessage(), 500, 'generate_failed');
```

```
        }
    }

    /**
     * 内容审核
     * @param Request $request
     * @return \think\Response
     */
    public function audit(Request $request)
    {
        $data = $request->post();

        try {
            $this->validate($data, [
                'content_id' => 'require|number',
                'action' => 'require|in:approve,reject',
                'reason' => 'max:500'
            ]);

            $result = $this->contentService->auditContent(
                $data['content_id'],
                $data['action'],
                $data['reason'] ?? '',
                $this->request->user_id
            );

            return $this->success($result, '审核完成');
        } catch (\Exception $e) {
            return $this->error($e->getMessage(), 500, 'audit_failed');
        }
    }
}
```

3.3 设备管理模块

3.3.1 功能概述

设备管理模块负责NFC设备和WiFi探针设备的管理，包括设备注册、状态监控、配置管理、触发规则设置等功能。

3.3.2 核心功能

1. **NFC设备管理**
- 设备注册与绑定
- 设备状态监控
- 批量操作支持

2. **WiFi探针管理**
- 探针设备配置
- MAC地址采集
- 到访记录管理

3. **触发规则配置**
- 自定义触发条件
- 多动作执行
- 规则优先级管理

3.3.3 技术实现

NFC服务实现:

```php
namespace app\service;

use app\model\NfcDevice;
use app\model\DeviceTrigger;
use think\facade\Cache;

class NfcService
{
    /**
     * 注册NFC设备
     * @param array $data
     * @return array
     */
    public function registerDevice(array $data): array
    {
        // 生成唯一设备ID
        $deviceId = $this->generateDeviceId();

        $device = new NfcDevice();
        $device->device_id = $deviceId;
        $device->merchant_id = $data['merchant_id'];
        $device->device_name = $data['device_name'];
        $device->device_type = $data['device_type'] ?? 'nfc_tag';
        $device->location = $data['location'] ?? '';
        $device->description = $data['description'] ?? '';
        $device->status = 'active';
        $device->save();

        // 创建默认触发规则
        $this->createDefaultTrigger($device->id, $data['merchant_id']);

        // 清除缓存
        Cache::delete('merchant_devices:' . $data['merchant_id']);

        return [
            'device_id' => $deviceId,
            'device_uuid' => $device->id
        ];
    }

    /**
     * 处理NFC触发事件
     * @param string $deviceId
     * @param array $triggerData
```

```
     * @return bool
     */
    public function handleTrigger(string $deviceId, array $triggerData): bool
    {
        // 获取设备信息
        $device = NfcDevice::where('device_id', $deviceId)
            ->where('status', 'active')
            ->find();

        if (!$device) {
            return false;
        }

        // 记录触发日志
        $this->logTrigger($device->id, $triggerData);

        // 获取触发规则
        $triggers = DeviceTrigger::where('device_id', $device->id)
            ->where('status', 'active')
            ->order('priority', 'desc')
            ->select();

        foreach ($triggers as $trigger) {
            if ($this->checkTriggerCondition($trigger, $triggerData)) {
                $this->executeTriggerAction($trigger, $triggerData);
            }
        }

        return true;
    }

    /**
     * 检查触发条件
     * @param DeviceTrigger $trigger
     * @param array $data
     * @return bool
     */
    private function checkTriggerCondition(DeviceTrigger $trigger, array $data): bool
    {
        $conditions = json_decode($trigger->conditions, true);

        // 时间条件检查
        if (isset($conditions['time'])) {
            $currentTime = date('H:i');
```

```
            $startTime = $conditions['time']['start'] ?? '00:00';
            $endTime = $conditions['time']['end'] ?? '23:59';

            if ($currentTime < $startTime || $currentTime > $endTime) {
                return false;
            }
        }

        // 频次条件检查
        if (isset($conditions['frequency'])) {
            $cacheKey = "trigger_freq:{$trigger->id}:{$data['user_id'] ?? 'anonymous'}";
            $count = Cache::get($cacheKey, 0);

            if ($count >= $conditions['frequency']['max_count']) {
                return false;
            }

            Cache::inc($cacheKey);
            Cache::expire($cacheKey, $conditions['frequency']['period'] ?? 3600);
        }

        return true;
    }

    /**
     * 执行触发动作
     * @param DeviceTrigger $trigger
     * @param array $data
     */
    private function executeTriggerAction(DeviceTrigger $trigger, array $data): void
    {
        $actions = json_decode($trigger->actions, true);

        foreach ($actions as $action) {
            switch ($action['type']) {
                case 'send_coupon':
                    $this->sendCoupon($action['coupon_id'], $data);
                    break;

                case 'show_content':
                    $this->showContent($action['content_id'], $data);
                    break;

                case 'open_url':
                    $this->openUrl($action['url'], $data);
```

```
                    break;

                case 'send_notification':
                    $this->sendNotification($action['message'], $data);
                    break;
            }
        }
    }
}
```

3.4 营销推广模块

3.4.1 功能概述

营销推广模块提供多平台内容发布、定时发布、营销活动管理、推荐系统等功能，帮助商户实现精准营销。

3.4.2 核心功能

1. **多平台发布**
- 微信生态
- 抖音平台
- 快手平台
- 自定义平台扩展

2. **定时发布**
- 定时任务管理
- 批量发布
- 发布状态跟踪

3. **推荐系统**
- 基于用户行为推荐
- 协同过滤算法
- 实时推荐更新

4. **营销活动**
- 优惠券发放
- 拼团活动

- 秒杀活动
- 会员活动

3.4.3 技术实现

发布服务实现:

```php
namespace app\service;

use app\model\PublishTask;
use app\service\DouyinService;
use app\service\WechatService;
use think\facade\Queue;

class PublishService
{
    private DouyinService $douyinService;
    private WechatService $wechatService;

    /**
     * 创建发布任务
     * @param array $data
     * @return int
     */
    public function createTask(array $data): int
    {
        $task = new PublishTask();
        $task->user_id = $data['user_id'];
        $task->merchant_id = $data['merchant_id'];
        $task->platform = $data['platform'];
        $task->content_type = $data['content_type'];
        $task->title = $data['title'];
        $task->content = $data['content'];
        $task->media_urls = $data['media_urls'] ?? [];
        $task->publish_time = $data['publish_time'] ?? null;
        $task->status = 'pending';
        $task->save();

        // 如果是定时发布，加入队列
        if ($task->publish_time && $task->publish_time > time()) {
            Queue::later(
                $task->publish_time - time(),
                'app\job\PublishJob@execute',
                ['task_id' => $task->id]
            );
        } else {
            // 立即发布
            Queue::push('app\job\PublishJob@execute', ['task_id' => $task->id]);
        }
```

```
        return $task->id;
    }

    /**
     * 执行发布任务
     * @param int $taskId
     * @return bool
     */
    public function executeTask(int $taskId): bool
    {
        $task = PublishTask::find($taskId);
        if (!$task || $task->status !== 'pending') {
            return false;
        }

        $task->status = 'publishing';
        $task->save();

        try {
            switch ($task->platform) {
                case 'douyin':
                    $result = $this->publishToDouyin($task);
                    break;

                case 'wechat':
                    $result = $this->publishToWechat($task);
                    break;

                case 'kuaishou':
                    $result = $this->publishToKuaishou($task);
                    break;

                default:
                    throw new \Exception('不支持的平台');
            }

            $task->status = 'success';
            $task->result = $result;
            $task->published_at = date('Y-m-d H:i:s');
            $task->save();

            return true;
        } catch (\Exception $e) {
            $task->status = 'failed';
```

```
            $task->error_message = $e->getMessage();
            $task->save();

            return false;
        }
    }

    /**
     * 发布到抖音
     * @param PublishTask $task
     * @return array
     */
    private function publishToDouyin(PublishTask $task): array
    {
        // 获取账号信息
        $account = PlatformAccount::where('user_id', $task->user_id)
            ->where('platform', 'douyin')
            ->where('status', 'active')
            ->find();

        if (!$account) {
            throw new \Exception('未找到有效的抖音账号');
        }

        // 调用抖音服务发布
        return $this->douyinService->publishVideo([
            'title' => $task->title,
            'video_path' => $task->media_urls[0] ?? '',
            'cover_path' => $task->media_urls[1] ?? '',
            'tags' => $task->tags ?? [],
            'access_token' => $account->access_token
        ]);
    }
}
```

3.5 数据分析模块

3.5.1 功能概述

数据分析模块提供用户行为分析、营销效果分析、实时数据展示、报表生成等功能，为商户决策提供数据支持。

3.5.2 核心功能

1. **用户行为分析**
- 访问轨迹追踪
- 行为漏斗分析
- 留存分析
- 转化路径分析

2. **营销效果分析**
- 曝光量统计
- 点击率分析
- 转化率计算
- ROI分析

3. **实时数据展示**
- 实时访问统计
- 实时转化监控
- 异常数据告警

4. **报表生成**
- 自定义报表
- 定时报表推送
- 数据导出功能

3.5.3 技术实现

数据分析服务实现:

```
namespace app\service;

use app\model\Statistics;
use think\facade\Db;
use think\facade\Cache;

class DataAnalysisService
{
    /**
     * 统计用户行为
     * @param array $data
     * @return bool
     */
    public function trackUserBehavior(array $data): bool
    {
        $statistics = new Statistics();
        $statistics->event_type = $data['event_type'];
        $statistics->user_id = $data['user_id'] ?? null;
        $statistics->merchant_id = $data['merchant_id'];
        $statistics->content_id = $data['content_id'] ?? null;
        $statistics->platform = $data['platform'];
        $statistics->device_info = $data['device_info'] ?? [];
        $statistics->extra_data = $data['extra_data'] ?? [];
        $statistics->created_at = date('Y-m-d H:i:s');
        $statistics->save();

        // 更新实时统计缓存
        $this->updateRealtimeStats($data);

        return true;
    }

    /**
     * 获取营销效果数据
     * @param int $merchantId
     * @param string $startDate
     * @param string $endDate
     * @return array
     */
    public function getMarketingStats(int $merchantId, string $startDate, string $endDat
    {
        // 曝光量统计
        $exposure = Statistics::where('merchant_id', $merchantId)
```

```
            ->where('event_type', 'exposure')
            ->whereBetween('created_at', [$startDate, $endDate])
            ->count();

        // 点击量统计
        $click = Statistics::where('merchant_id', $merchantId)
            ->where('event_type', 'click')
            ->whereBetween('created_at', [$startDate, $endDate])
            ->count();

        // 转化量统计
        $conversion = Statistics::where('merchant_id', $merchantId)
            ->where('event_type', 'conversion')
            ->whereBetween('created_at', [$startDate, $endDate])
            ->count();

        // 计算率值
        $ctr = $exposure > 0 ? round(($click / $exposure) * 100, 2) : 0;
        $cvr = $click > 0 ? round(($conversion / $click) * 100, 2) : 0;

        // 趋势数据
        $trend = $this->getMarketingTrend($merchantId, $startDate, $endDate);

        return [
            'exposure' => $exposure,
            'click' => $click,
            'conversion' => $conversion,
            'ctr' => $ctr,
            'cvr' => $cvr,
            'trend' => $trend
        ];
    }

    /**
     * 生成数据分析报表
     * @param int $merchantId
     * @param array $params
     * @return array
     */
    public function generateReport(int $merchantId, array $params): array
    {
        $reportType = $params['report_type'] ?? 'daily';
        $startDate = $params['start_date'] ?? date('Y-m-d', strtotime('-30 days'));
        $endDate = $params['end_date'] ?? date('Y-m-d');
```

```
        switch ($reportType) {
            case 'daily':
                return $this->generateDailyReport($merchantId, $startDate, $endDate);

            case 'weekly':
                return $this->generateWeeklyReport($merchantId, $startDate, $endDate);

            case 'monthly':
                return $this->generateMonthlyReport($merchantId, $startDate, $endDate);

            case 'campaign':
                return $this->generateCampaignReport($merchantId, $params);

            default:
                throw new \Exception('不支持的报表类型');
        }
    }

    /**
     * 生成日报表
     * @param int $merchantId
     * @param string $startDate
     * @param string $endDate
     * @return array
     */
    private function generateDailyReport(int $merchantId, string $startDate, string $end
    {
        $report = [];
        $currentDate = $startDate;

        while ($currentDate <= $endDate) {
            $dayStats = $this->getMarketingStats(
                $merchantId,
                $currentDate . ' 00:00:00',
                $currentDate . ' 23:59:59'
            );

            $report[] = [
                'date' => $currentDate,
                'exposure' => $dayStats['exposure'],
                'click' => $dayStats['click'],
                'conversion' => $dayStats['conversion'],
                'ctr' => $dayStats['ctr'],
                'cvr' => $dayStats['cvr']
            ];
```

```
            $currentDate = date('Y-m-d', strtotime($currentDate . ' +1 day'));
        }

        return $report;
    }
}
```

3.6 商户管理模块

3.6.1 功能概述

商户管理模块负责商户信息管理、联系方式配置、服务记录管理等功能，是系统的基础管理模块。

3.6.2 核心功能

1. **商户信息管理**
- 基本信息维护
- 资质认证
- 账号管理

2. **联系方式配置**
- 多联系方式支持
- 联系状态设置
- 智能路由

3. **服务记录管理**
- 服务历史记录
- 服务评价
- 服务统计

3.6.3 技术实现

商户服务实现:

```php
namespace app\service;

use app\model\Merchant;
use app\model\ServiceCall;
use think\facade\Db;

class MerchantService
{
    /**
     * 创建商户
     * @param array $data
     * @return Merchant
     */
    public function createMerchant(array $data): Merchant
    {
        Db::startTrans();
        try {
            $merchant = new Merchant();
            $merchant->merchant_name = $data['merchant_name'];
            $merchant->industry = $data['industry'];
            $merchant->address = $data['address'] ?? '';
            $merchant->contact_config = json_encode($data['contact_config'] ?? []);
            $merchant->business_hours = $data['business_hours'] ?? '';
            $merchant->description = $data['description'] ?? '';
            $merchant->status = 'active';
            $merchant->save();

            // 初始化商户配置
            $this->initMerchantConfig($merchant->id);

            Db::commit();
            return $merchant;
        } catch (\Exception $e) {
            Db::rollback();
            throw $e;
        }
    }

    /**
     * 更新商户信息
     * @param int $merchantId
     * @param array $data
     * @return bool
```

```
     */
    public function updateMerchant(int $merchantId, array $data): bool
    {
        $merchant = Merchant::find($merchantId);
        if (!$merchant) {
            throw new \Exception('商户不存在');
        }

        $allowedFields = [
            'merchant_name', 'industry', 'address',
            'contact_config', 'business_hours', 'description'
        ];

        foreach ($allowedFields as $field) {
            if (isset($data[$field])) {
                $merchant->$field = is_array($data[$field])
                    ? json_encode($data[$field])
                    : $data[$field];
            }
        }

        return $merchant->save();
    }

    /**
     * 记录服务调用
     * @param array $data
     * @return ServiceCall
     */
    public function recordServiceCall(array $data): ServiceCall
    {
        $call = new ServiceCall();
        $call->merchant_id = $data['merchant_id'];
        $call->user_id = $data['user_id'] ?? null;
        $call->contact_type = $data['contact_type'];
        $call->contact_value = $data['contact_value'];
        $call->call_status = $data['call_status'] ?? 'initiated';
        $call->duration = $data['duration'] ?? 0;
        $call->extra_data = $data['extra_data'] ?? [];
        $call->created_at = date('Y-m-d H:i:s');
        $call->save();

        return $call;
    }
```

```
}
```

第4章 技术实现细节

4.1 后端架构实现

4.1.1 MVC架构设计

系统采用标准的MVC（Model-View-Controller）架构模式，结合ThinkPHP 8.0框架特性，实现了清晰的分层结构。

目录结构:

```
api/
├── app/
│   ├── controller/     # 控制器层
│   │   ├── Auth.php
│   │   ├── Content.php
│   │   ├── DeviceManage.php
│   │   └── ...
│   ├── model/          # 模型层
│   │   ├── User.php
│   │   ├── Merchant.php
│   │   └── ...
│   ├── service/        # 服务层
│   │   ├── AuthService.php
│   │   ├── ContentService.php
│   │   └── ...
│   ├── middleware/     # 中间件
│   ├── validate/       # 验证器
│   └── job/           # 任务队列
├── config/            # 配置文件
├── route/             # 路由配置
└── public/            # 入口文件
```

4.1.2 控制器基类实现

```
namespace app\controller;

use think\App;
use think\exception\ValidateException;
use think\Response;

abstract class BaseController
{
    protected App $app;
    protected Request $request;

    /**
     * 构造函数
     * @param App $app
     */
    public function __construct(App $app)
    {
        $this->app = $app;
        $this->request = $this->app->request;

        // 控制器初始化
        $this->initialize();
    }

    /**
     * 初始化
     */
    protected function initialize(): void
    {
        // 子类可重写此方法
    }

    /**
     * 成功响应
     * @param mixed $data
     * @param string $msg
     * @param int $code
     * @return Response
     */
    protected function success($data = null, string $msg = 'success', int $code = 200):
    {
        $result = [
            'code' => $code,
            'msg' => $msg,
```

```
            'time' => time(),
            'data' => $data
        ];

        return Response::create($result, 'json', $code);
    }

    /**
     * 错误响应
     * @param string $msg
     * @param int $code
     * @param string $error_code
     * @return Response
     */
    protected function error(string $msg = 'error', int $code = 400, string $error_code
    {
        $result = [
            'code' => $code,
            'msg' => $msg,
            'time' => time(),
            'data' => null
        ];

        if ($error_code) {
            $result['error_code'] = $error_code;
        }

        return Response::create($result, 'json', $code);
    }
}
```

4.2 数据库设计

4.2.1 数据库设计原则

1. **命名规范**
- 表名：xmt_前缀 + 小写字母 + 下划线分隔
- 字段名：小写字母 + 下划线分隔
- 索引名：idx_前缀 + 字段名

2. **字段规范**
- 主键：统一使用id，自增
- 时间戳：create_time、update_time
- 软删除：delete_time
- 状态：status字段，使用枚举值

3. **性能优化**
- 合理使用索引
- 避免大字段
- 定期归档历史数据

4.2.2 核心表设计

用户表(xmt_users):

```
CREATE TABLE xmt_users (
  id bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '用户ID',
  openid varchar(128) NOT NULL DEFAULT '' COMMENT '微信openid',
  unionid varchar(128) NOT NULL DEFAULT '' COMMENT '微信unionid',
  phone varchar(20) NOT NULL DEFAULT '' COMMENT '手机号',
  nickname varchar(50) NOT NULL DEFAULT '' COMMENT '昵称',
  avatar varchar(255) NOT NULL DEFAULT '' COMMENT '头像URL',
  gender tinyint(1) NOT NULL DEFAULT 0 COMMENT '性别：0未知 1男 2女',
  city varchar(50) NOT NULL DEFAULT '' COMMENT '城市',
  province varchar(50) NOT NULL DEFAULT '' COMMENT '省份',
  country varchar(50) NOT NULL DEFAULT '' COMMENT '国家',
  user_type varchar(20) NOT NULL DEFAULT 'user' COMMENT '用户类型',
  status tinyint(1) NOT NULL DEFAULT 1 COMMENT '状态：0禁用 1正常',
  last_login_time datetime DEFAULT NULL COMMENT '最后登录时间',
  last_login_ip varchar(50) DEFAULT '' COMMENT '最后登录IP',
  created_at datetime DEFAULT NULL COMMENT '创建时间',
  updated_at datetime DEFAULT NULL COMMENT '更新时间',
  deleted_at datetime DEFAULT NULL COMMENT '删除时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_openid (openid),
  KEY idx_phone (phone),
  KEY idx_user_type (user_type),
  KEY idx_status (status),
  KEY idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户表';
```

商户表(xmt_merchants):

```
CREATE TABLE xmt_merchants (
  id bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '商户ID',
  merchant_no varchar(32) NOT NULL DEFAULT '' COMMENT '商户编号',
  merchant_name varchar(100) NOT NULL DEFAULT '' COMMENT '商户名称',
  industry varchar(50) NOT NULL DEFAULT '' COMMENT '所属行业',
  address varchar(255) NOT NULL DEFAULT '' COMMENT '详细地址',
  longitude decimal(10,7) NOT NULL DEFAULT 0.0000000 COMMENT '经度',
  latitude decimal(10,7) NOT NULL DEFAULT 0.0000000 COMMENT '纬度',
  contact_config json DEFAULT NULL COMMENT '联系方式配置',
  business_hours varchar(100) NOT NULL DEFAULT '' COMMENT '营业时间',
  description text COMMENT '商户描述',
  logo varchar(255) NOT NULL DEFAULT '' COMMENT '商户logo',
  certificates json DEFAULT NULL COMMENT '资质证书',
  status tinyint(1) NOT NULL DEFAULT 1 COMMENT '状态：0禁用 1正常',
  created_at datetime DEFAULT NULL COMMENT '创建时间',
  updated_at datetime DEFAULT NULL COMMENT '更新时间',
  deleted_at datetime DEFAULT NULL COMMENT '删除时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_merchant_no (merchant_no),
  KEY idx_industry (industry),
  KEY idx_status (status),
  KEY idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='商户表';
```

内容任务表(xmt_content_tasks):

```sql
CREATE TABLE xmt_content_tasks (
  id bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '任务ID',
  user_id bigint(20) unsigned NOT NULL COMMENT '用户ID',
  merchant_id bigint(20) unsigned NOT NULL COMMENT '商户ID',
  task_type varchar(20) NOT NULL DEFAULT 'text' COMMENT '任务类型',
  title varchar(200) NOT NULL DEFAULT '' COMMENT '标题',
  content longtext COMMENT '内容',
  prompt text COMMENT '生成提示词',
  params json DEFAULT NULL COMMENT '生成参数',
  result json DEFAULT NULL COMMENT '生成结果',
  status varchar(20) NOT NULL DEFAULT 'pending' COMMENT '状态',
  error_message text COMMENT '错误信息',
  created_at datetime DEFAULT NULL COMMENT '创建时间',
  updated_at datetime DEFAULT NULL COMMENT '更新时间',
  PRIMARY KEY (id),
  KEY idx_user_id (user_id),
  KEY idx_merchant_id (merchant_id),
  KEY idx_task_type (task_type),
  KEY idx_status (status),
  KEY idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='内容生成任务表
```

4.2.3 索引优化策略

1. **主键索引**
- 所有表使用自增主键
- 避免使用UUID作为主键

2. **唯一索引**
- openId、phone等唯一字段
- 商户编号等业务唯一字段

3. **组合索引**
- 经常一起查询的字段
- 遵循最左前缀原则

4. **覆盖索引**
- 查询字段包含在索引中
- 减少回表查询

4.3 API接口设计

4.3.1 RESTful API设计规范

1. **URL设计**
- 使用名词复数形式
- 使用小写字母和连字符
- 版本控制：/api/v1/

2. **HTTP方法**
- GET：查询资源
- POST：创建资源
- PUT：更新资源
- DELETE：删除资源

3. **状态码规范**
- 200：成功
- 201：创建成功
- 400：请求错误
- 401：未授权
- 403：禁止访问
- 404：资源不存在
- 500：服务器错误

4.3.2 API接口示例

用户认证接口:

```
// 登录接口
POST /api/v1/auth/login
Content-Type: application/json

{
    "code": "wx_code",
    "encrypted_data": "encrypted_data",
    "iv": "iv"
}

Response:
{
    "code": 200,
    "msg": "登录成功",
    "time": 1703123456,
    "data": {
        "user_id": 1001,
        "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
        "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
        "expires_in": 7200,
        "user_info": {
            "nickname": "小明",
            "avatar": "https://...",
            "phone": "13800138000"
        }
    }
}
```

内容生成接口:

```
POST /api/v1/content/generate
Authorization: Bearer {access_token}
Content-Type: application/json

{
    "type": "text",
    "prompt": "为奶茶店写一个促销文案",
    "params": {
        "industry": "餐饮",
        "product": "珍珠奶茶",
        "style": "活泼",
        "length": "短",
        "temperature": 0.8
    }
}

Response:
{
    "code": 200,
    "msg": "生成成功",
    "time": 1703123456,
    "data": {
        "content_id": 10001,
        "content": "【限时特惠】夏日炎炎，来一杯冰镇珍珠奶茶！第二杯半价，和朋友一起分享清凉~",
        "usage": {
            "prompt_tokens": 50,
            "completion_tokens": 100,
            "total_tokens": 150
        }
    }
}
```

4.3.3 API文档生成

使用Swagger/OpenAPI 3.0规范生成API文档：

```yaml
openapi: 3.0.0
info:
  title: 小磨推智能营销系统API
  version: 1.0.0
  description: 提供智能营销、内容生成、数据分析等功能的RESTful API

paths:
  /api/v1/auth/login:
    post:
      summary: 用户登录
      tags:
        - 认证模块
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                code:
                  type: string
                  description: 微信小程序code
                encrypted_data:
                  type: string
                  description: 加密数据
                iv:
                  type: string
                  description: 初始向量
      responses:
        '200':
          description: 登录成功
          content:
            application/json:
              schema:
                type: object
                properties:
                  code:
                    type: integer
                  msg:
                    type: string
                  data:
                    type: object
```

```
                    properties:
                      access_token:
                        type: string
                      refresh_token:
                        type: string
                      expires_in:
                        type: integer
```

4.4 安全机制实现

4.4.1 JWT认证机制

```php
namespace app\service;

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

class AuthService
{
    private string $secretKey;
    private int $accessTokenExpire = 7200;  // 2小时
    private int $refreshTokenExpire = 604800; // 7天

    public function __construct()
    {
        $this->secretKey = config('jwt.secret_key');
    }

    /**
     * 生成访问令牌
     * @param array $user
     * @return array
     */
    public function generateTokens(array $user): array
    {
        $now = time();

        // 访问令牌
        $accessPayload = [
            'iss' => 'xiaomotui',
            'aud' => 'xiaomotui-app',
            'iat' => $now,
            'exp' => $now + $this->accessTokenExpire,
            'user_id' => $user['id'],
            'user_type' => $user['user_type'] ?? 'user',
            'scope' => 'access'
        ];

        // 刷新令牌
        $refreshPayload = [
            'iss' => 'xiaomotui',
            'aud' => 'xiaomotui-app',
            'iat' => $now,
            'exp' => $now + $this->refreshTokenExpire,
            'user_id' => $user['id'],
```

```
            'scope' => 'refresh'
        ];

        $accessToken = JWT::encode($accessPayload, $this->secretKey, 'HS256');
        $refreshToken = JWT::encode($refreshPayload, $this->secretKey, 'HS256');

        // 存储刷新令牌到缓存
        cache('refresh_token:' . $user['id'], $refreshToken, $this->refreshTokenExpire);

        return [
            'access_token' => $accessToken,
            'refresh_token' => $refreshToken,
            'expires_in' => $this->accessTokenExpire
        ];
    }

    /**
     * 验证令牌
     * @param string $token
     * @return array
     */
    public function verifyToken(string $token): array
    {
        try {
            $payload = JWT::decode($token, new Key($this->secretKey, 'HS256'));
            return (array)$payload;
        } catch (\Exception $e) {
            throw new \Exception('令牌无效或已过期');
        }
    }
}
```

4.4.2 数据加密机制

```php
namespace app\common\utils;

class EncryptionUtil
{
    /**
     * AES加密
     * @param string $data
     * @param string $key
     * @return string
     */
    public static function aesEncrypt(string $data, string $key): string
    {
        $iv = random_bytes(openssl_cipher_iv_length('aes-256-cbc'));
        $encrypted = openssl_encrypt($data, 'aes-256-cbc', $key, 0, $iv);
        return base64_encode($iv . $encrypted);
    }

    /**
     * AES解密
     * @param string $data
     * @param string $key
     * @return string
     */
    public static function aesDecrypt(string $data, string $key): string
    {
        $data = base64_decode($data);
        $ivLength = openssl_cipher_iv_length('aes-256-cbc');
        $iv = substr($data, 0, $ivLength);
        $encrypted = substr($data, $ivLength);
        return openssl_decrypt($encrypted, 'aes-256-cbc', $key, 0, $iv);
    }

    /**
     * 密码哈希
     * @param string $password
     * @return string
     */
    public static function hashPassword(string $password): string
    {
        return password_hash($password, PASSWORD_ARGON2ID, [
            'memory_cost' => 65536,
            'time_cost' => 4,
            'threads' => 3
```

```
        ]);
    }

    /**
     * 验证密码
     * @param string $password
     * @param string $hash
     * @return bool
     */
    public static function verifyPassword(string $password, string $hash): bool
    {
        return password_verify($password, $hash);
    }
}
```

4.5 缓存策略

4.5.1 Redis缓存配置

```
// config/cache.php
return [
    'default' => env('cache.driver', 'redis'),

    'stores' => [
        'redis' => [
            'type' => 'redis',
            'host' => env('redis.host', '127.0.0.1'),
            'port' => env('redis.port', 6379),
            'password' => env('redis.password', ''),
            'select' => env('redis.select', 0),
            'timeout' => 0,
            'expire' => 0,
            'persistent' => false,
            'prefix' => 'xiaomotui:',
        ],
    ],
];
```

4.5.2 缓存服务实现

```
namespace app\service;

use think\facade\Cache;

class CacheService
{
    /**
     * 缓存用户信息
     * @param int $userId
     * @param array $userInfo
     * @param int $expire
     */
    public static function cacheUserInfo(int $userId, array $userInfo, int $expire = 360
    {
        $key = "user:info:{$userId}";
        Cache::set($key, $userInfo, $expire);
    }

    /**
     * 获取缓存的用户信息
     * @param int $userId
     * @return array|null
     */
    public static function getUserInfo(int $userId): ?array
    {
        $key = "user:info:{$userId}";
        return Cache::get($key);
    }

    /**
     * 缓存商户设备列表
     * @param int $merchantId
     * @param array $devices
     * @param int $expire
     */
    public static function cacheMerchantDevices(int $merchantId, array $devices, int $ex
    {
        $key = "merchant:devices:{$merchantId}";
        Cache::set($key, $devices, $expire);
    }

    /**
     * 记录API访问次数
```

```
     * @param string $key
     * @param int $window
     * @return int
     */
    public static function incrApiAccess(string $key, int $window = 60): int
    {
        $count = Cache::inc($key);
        if ($count === 1) {
            Cache::expire($key, $window);
        }
        return $count;
    }
}
```

4.6 消息队列

4.6.1 队列配置

```
// config/queue.php
return [
    'default' => env('queue.driver', 'redis'),

    'connections' => [
        'redis' => [
            'type' => 'redis',
            'host' => env('redis.host', '127.0.0.1'),
            'port' => env('redis.port', 6379),
            'password' => env('redis.password', ''),
            'select' => env('redis.select', 1),
            'queue' => 'default',
        ],
    ],
];
```

4.6.2 任务队列实现

```php
namespace app\job;

use think\queue\Job;
use app\service\PublishService;

class PublishJob
{
    /**
     * 执行队列任务
     * @param Job $job
     * @param array $data
     */
    public function execute(Job $job, array $data): void
    {
        try {
            $publishService = new PublishService();
            $result = $publishService->executeTask($data['task_id']);

            if ($result) {
                $job->delete();
            } else {
                // 失败重试
                if ($job->attempts() < 3) {
                    $job->release(60);
                } else {
                    $job->delete();
                }
            }
        } catch (\Exception $e) {
            // 记录错误日志
            \think\facade\Log::error('Publish task failed: ' . $e->getMessage());

            // 失败重试
            if ($job->attempts() < 3) {
                $job->release(60);
            } else {
                $job->delete();
            }
        }
    }

    /**
     * 任务失败处理
```

```
     * @param array $data
     */
    public function failed(array $data): void
    {
        // 更新任务状态为失败
        Db::table('xmt_publish_tasks')
            ->where('id', $data['task_id'])
            ->update([
                'status' => 'failed',
                'error_message' => '队列任务执行失败'
            ]);
    }
}
```

第5章 性能优化

5.1 数据库优化

5.1.1 查询优化

1. **避免全表扫描**
- 合理使用索引
- 避免在WHERE子句中对字段进行函数操作
- 使用EXPLAIN分析查询计划

2. **优化查询语句**

```
// 优化前
$users = Db::table('users')
    ->where('YEAR(created_at)', 2024)
    ->select();

// 优化后
$users = Db::table('users')
    ->whereBetween('created_at', ['2024-01-01', '2024-12-31'])
    ->select();
```

3. **批量操作优化**

```
// 优化前：循环插入
foreach ($dataList as $data) {
    Db::table('table')->insert($data);
}

// 优化后：批量插入
Db::table('table')->insertAll($dataList);
```

5.1.2 连接池优化

```
// database.php中的连接池配置
'pool' => [
    'min_connections' => 5,
    'max_connections' => 20,
    'connect_timeout' => 10.0,
    'wait_timeout' => 3.0,
    'heartbeat' => 60,
    'max_idle_time' => 60,
],
```

5.1.3 读写分离配置

```
// 主库配置
'master' => [
    'hostname' => 'master.db.com',
    'username' => 'write_user',
    'password' => 'write_pass',
],

// 从库配置
'slave' => [
    'hostname' => 'slave.db.com',
    'username' => 'read_user',
    'password' => 'read_pass',
],

// 使用示例
// 写操作自动使用主库
Db::name('user')->insert($data);

// 读操作自动使用从库
$users = Db::name('user')->select();
```

5.2 缓存优化

5.2.1 多级缓存策略

```php
namespace app\service;

class CacheService
{
    /**
     * 多级缓存获取
     * @param string $key
     * @param callable $callback
     * @param int $l1Expire L1缓存(内存)过期时间
     * @param int $l2Expire L2缓存(Redis)过期时间
     * @return mixed
     */
    public static function remember(string $key, callable $callback, int $l1Expire = 60,
    {
        // L1缓存：APCu/OPcache
        $l1Key = "l1:{$key}";
        if (function_exists('apcu_fetch')) {
            $value = apcu_fetch($l1Key);
            if ($value !== false) {
                return $value;
            }
        }

        // L2缓存：Redis
        $value = Cache::get($key);
        if ($value !== null) {
            // 回写L1缓存
            if (function_exists('apcu_store')) {
                apcu_store($l1Key, $value, $l1Expire);
            }
            return $value;
        }

        // 缓存未命中，执行回调获取数据
        $value = $callback();

        // 写入缓存
        Cache::set($key, $value, $l2Expire);
        if (function_exists('apcu_store')) {
            apcu_store($l1Key, $value, $l1Expire);
        }
```

```
        return $value;
    }
}
```

5.2.2 缓存预热策略

```php
namespace app\command;

use think\console\Command;
use think\console\Input;
use think\console\Output;

class CacheWarmup extends Command
{
    protected function configure()
    {
        $this->setName('cache:warmup')
            ->setDescription('缓存预热');
    }

    protected function execute(Input $input, Output $output)
    {
        // 预热热点数据
        $this->warmupHotData();

        // 预热配置数据
        $this->warmupConfigData();

        // 预热商户数据
        $this->warmupMerchantData();

        $output->writeln('缓存预热完成');
    }

    private function warmupHotData(): void
    {
        // 预热热门内容
        $hotContents = Db::name('content')
            ->where('status', 'active')
            ->order('views', 'desc')
            ->limit(100)
            ->select();

        foreach ($hotContents as $content) {
            CacheService::cacheContent($content['id'], $content);
        }
    }
}
```

5.3 接口性能优化

5.3.1 响应压缩

```
// middleware.php中启用压缩
\think\middleware\AllowCrossDomain::class,
\think\middleware\CheckResponseCache::class,
\think\middleware\CompressResponse::class, // 自定义压缩中间件
```

5.3.2 限流控制

```
namespace app\middleware;

use think\Request;
use think\Response;

class RateLimit
{
    public function handle(Request $request, \Closure $next)
    {
        $key = 'rate_limit:' . $request->ip();
        $limit = 100; // 每分钟100次
        $window = 60;

        $count = CacheService::incrApiAccess($key, $window);

        if ($count > $limit) {
            return Response::create([
                'code' => 429,
                'msg' => '请求过于频繁，请稍后再试'
            ], 'json', 429);
        }

        return $next($request);
    }
}
```

5.4 前端性能优化

5.4.1 资源优化

1. **图片优化**
- WebP格式转换
- 懒加载实现
- CDN加速

2. **代码优化**
- JS/CSS压缩
- Tree Shaking
- 代码分割

```
// Webpack配置示例
module.exports = {
    optimization: {
        splitChunks: {
            chunks: 'all',
            minSize: 30000,
            maxSize: 0,
            minChunks: 1,
            maxAsyncRequests: 5,
            maxInitialRequests: 3,
            automaticNameDelimiter: '~',
            cacheGroups: {
                vendors: {
                    test: /[\\/]node_modules[\\/]/,
                    priority: -10
                },
                default: {
                    minChunks: 2,
                    priority: -20,
                    reuseExistingChunk: true
                }
            }
        }
    }
}
```

第6章 安全机制

6.1 身份认证

6.1.1 多因子认证

```
namespace app\service;

class MfaService
{
    /**
     * 发送短信验证码
     * @param string $phone
     * @return string
     */
    public function sendSmsCode(string $phone): string
    {
        $code = $this->generateCode();
        $cacheKey = "sms_code:{$phone}";

        // 存储验证码
        Cache::set($cacheKey, [
            'code' => $code,
            'attempts' => 0,
            'sent_at' => time()
        ], 300);

        // 调用短信服务
        $this->smsService->send($phone, "您的验证码是：{$code}，5分钟内有效");

        return $code;
    }

    /**
     * 验证短信验证码
     * @param string $phone
     * @param string $code
     * @return bool
     */
    public function verifySmsCode(string $phone, string $code): bool
    {
        $cacheKey = "sms_code:{$phone}";
        $data = Cache::get($cacheKey);

        if (!$data) {
            return false;
        }

        // 检查尝试次数
        if ($data['attempts'] >= 5) {
```

```
            Cache::delete($cacheKey);
            return false;
        }

        // 验证码错误
        if ($data['code'] !== $code) {
            $data['attempts']++;
            Cache::set($cacheKey, $data, 300);
            return false;
        }

        // 验证成功，删除缓存
        Cache::delete($cacheKey);
        return true;
    }
}
```

6.2 权限控制

6.2.1 RBAC权限模型

```php
namespace app\model;

use think\Model;

class Permission extends Model
{
    protected $table = 'xmt_permissions';

    /**
     * 检查用户权限
     * @param int $userId
     * @param string $permission
     * @return bool
     */
    public static function checkPermission(int $userId, string $permission): bool
    {
        // 获取用户角色
        $roles = UserRole::where('user_id', $userId)
            ->column('role_id');

        if (empty($roles)) {
            return false;
        }

        // 获取角色权限
        $permissions = RolePermission::whereIn('role_id', $roles)
            ->alias('rp')
            ->join('permission p', 'rp.permission_id = p.id')
            ->where('p.code', $permission)
            ->count();

        return $permissions > 0;
    }
}
```

6.3 数据加密

6.3.1 敏感数据加密

```php
namespace app\service;

class EncryptionService
{
    private string $key;

    public function __construct()
    {
        $this->key = config('app.data_encryption_key');
    }

    /**
     * 加密敏感字段
     * @param string $data
     * @return string
     */
    public function encrypt(string $data): string
    {
        $nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
        $encrypted = sodium_crypto_secretbox($data, $nonce, $this->key);
        return base64_encode($nonce . $encrypted);
    }

    /**
     * 解密敏感字段
     * @param string $encrypted
     * @return string
     */
    public function decrypt(string $encrypted): string
    {
        $decoded = base64_decode($encrypted);
        $nonce = substr($decoded, 0, SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
        $ciphertext = substr($decoded, SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);

        $data = sodium_crypto_secretbox_open($ciphertext, $nonce, $this->key);
        if ($data === false) {
            throw new \Exception('解密失败');
        }

        return $data;
    }
}
```

6.4 安全审计

6.4.1 操作日志

```
namespace app\service;

class AuditService
{
    /**
     * 记录操作日志
     * @param int $userId
     * @param string $action
     * @param array $data
     * @param string $ip
     */
    public static function log(int $userId, string $action, array $data = [], string $ip
    {
        $log = [
            'user_id' => $userId,
            'action' => $action,
            'data' => json_encode($data),
            'ip' => $ip ?: request()->ip(),
            'user_agent' => request()->header('User-Agent'),
            'created_at' => date('Y-m-d H:i:s')
        ];

        Db::table('xmt_audit_logs')->insert($log);
    }

    /**
     * 异常行为检测
     * @param int $userId
     * @return array
     */
    public function detectAnomalies(int $userId): array
    {
        $anomalies = [];

        // 检测频繁登录
        $recentLogins = Db::table('xmt_audit_logs')
            ->where('user_id', $userId)
            ->where('action', 'login')
            ->where('created_at', '>', date('Y-m-d H:i:s', strtotime('-1 hour')))
            ->count();

        if ($recentLogins > 10) {
            $anomalies[] = '频繁登录异常';
```

```
        }

        // 检测异地登录
        $lastIp = Db::table('xmt_audit_logs')
            ->where('user_id', $userId)
            ->where('action', 'login')
            ->order('id', 'desc')
            ->value('ip');

        $currentIp = request()->ip();
        if ($lastIp && $this->isDifferentLocation($lastIp, $currentIp)) {
            $anomalies[] = '异地登录';
        }

        return $anomalies;
    }
}
```

第7章 部署与运维

7.1 环境要求

7.1.1 服务器要求

最低配置:
- CPU: 2核
- 内存: 4GB
- 硬盘: 50GB SSD
- 带宽: 5Mbps

推荐配置:
- CPU: 4核
- 内存: 8GB
- 硬盘: 100GB SSD
- 带宽: 10Mbps

生产环境:
- CPU: 8核

- 内存: 16GB
- 硬盘: 200GB SSD
- 带宽: 20Mbps

7.1.2 软件要求

- **操作系统**: CentOS 7.6+ / Ubuntu 18.04+
- **Web服务器**: Nginx 1.18+
- **PHP版本**: PHP 8.1+
- **数据库**: MySQL 8.0+ / MariaDB 10.5+
- **缓存**: Redis 6.0+
- **消息队列**: RabbitMQ 3.8+ / Redis

7.2 部署流程

7.2.1 自动化部署脚本

```
#!/bin/bash
```

deploy.sh - 自动化部署脚本

```
set -e
```

配置变量

```
PROJECT_NAME="xiaomotui"
DEPLOY_PATH="/var/www/$PROJECT_NAME"
BACKUP_PATH="/var/backups/$PROJECT_NAME"
GIT_REPO="https://github.com/your-repo/$PROJECT_NAME.git"
BRANCH="master"
```

创建备份

```
create_backup() {
    echo "创建备份..."
    mkdir -p $BACKUP_PATH
    tar -czf "$BACKUP_PATH/backup_$(date +%Y%m%d_%H%M%S).tar.gz" -C $DEPLOY_PATH .
}
```

更新代码

```
update_code() {
    echo "更新代码..."
    cd $DEPLOY_PATH
    git fetch origin
    git reset --hard origin/$BRANCH
}
```

安装依赖

```
install_dependencies() {
    echo "安装依赖..."
    cd $DEPLOY_PATH/api
    composer install --no-dev --optimize-autoloader

    cd $DEPLOY_PATH/uni-app
    npm install --production
}
```

执行数据库迁移

```
run_migrations() {
    echo "执行数据库迁移..."
    cd $DEPLOY_PATH/api
    php think migrate:run
}
```

清理缓存

```
clear_cache() {
    echo "清理缓存..."
    cd $DEPLOY_PATH/api
    php think clear
    php think optimize:route
    php think optimize:schema
}
```

重启服务

```
restart_services() {
    echo "重启服务..."
    supervisorctl restart xiaomotui-worker
    nginx -s reload
    php-fpm -y reload
}
```

执行部署

```
main() {
    create_backup
    update_code
    install_dependencies
    run_migrations
    clear_cache
    restart_services

    echo "部署完成！"
}

main "$@"
```

7.2.2 Docker部署

Dockerfile

```
FROM php:8.1-fpm-alpine
```

安装系统依赖

```
RUN apk add --no-cache \
    nginx \
    supervisor \
    mysql-client \
    curl \
    wget \
    git
```

安装PHP扩展

```
RUN docker-php-ext-install \
    pdo_mysql \
    mysqli \
    gd \
    zip \
    opcache \
    bcmath
```

安装Composer

```
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
```

复制应用代码

```
COPY . /var/www/xiaomotui
```

```
WORKDIR /var/www/xiaomotui
```

安装依赖

```
RUN composer install --no-dev --optimize-autoloader
```

配置文件

```
COPY docker/nginx.conf /etc/nginx/nginx.conf
COPY docker/php.ini /usr/local/etc/php/php.ini
COPY docker/supervisord.conf /etc/supervisord.conf
```

设置权限

```
RUN chown -R www-data:www-data /var/www/xiaomotui
```

暴露端口

```
EXPOSE 80 443
```

启动服务

```
CMD ["/usr/bin/supervisord", "-c", "/etc/supervisord.conf"]
```

docker-compose.yml

```
version: '3.8'

services:
  app:
    build: .
    ports:
      - "80:80"
      - "443:443"
    environment:
      - APP_ENV=production
      - DATABASE_HOST=mysql
      - REDIS_HOST=redis
    depends_on:
      - mysql
      - redis
    volumes:
      - ./uploads:/var/www/xiaomotui/uploads
      - ./logs:/var/log

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    ports:
      - "3306:3306"

  redis:
    image: redis:6.2-alpine
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data
    ports:
      - "6379:6379"
```

```
  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./docker/nginx.conf:/etc/nginx/nginx.conf
      - ./uploads:/var/www/uploads
    depends_on:
      - app

volumes:
  mysql_data:
  redis_data:
```

7.3 监控告警

7.3.1 Prometheus监控配置

prometheus.yml

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'xiaomotui-api'
    static_configs:
      - targets: ['localhost:9090']
    metrics_path: '/metrics'
    scrape_interval: 5s

  - job_name: 'nginx'
    static_configs:
      - targets: ['localhost:9113']

  - job_name: 'mysql'
    static_configs:
      - targets: ['localhost:9104']

  - job_name: 'redis'
    static_configs:
      - targets: ['localhost:9121']
```

7.3.2 Grafana监控面板

```
{
  "dashboard": {
    "title": "小磨推系统监控",
    "panels": [
      {
        "title": "API请求数",
        "type": "graph",
        "targets": [
          {
            "expr": "rate(http_requests_total[5m])",
            "legendFormat": "{{method}} {{route}}"
          }
        ]
      },
      {
        "title": "响应时间",
        "type": "graph",
        "targets": [
          {
            "expr": "histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[
            "legendFormat": "95th percentile"
          }
        ]
      },
      {
        "title": "数据库连接数",
        "type": "singlestat",
        "targets": [
          {
            "expr": "mysql_global_status_threads_connected"
          }
        ]
      }
    ]
  }
}
```

7.4 备份恢复

7.4.1 自动备份脚本

```
#!/bin/bash
```

backup.sh - 数据备份脚本

```
set -e
```

配置

```
BACKUP_DIR="/var/backups/xiaomotui"
DB_NAME="xiaomotui"
DB_USER="backup_user"
DB_PASS="backup_password"
S3_BUCKET="xiaomotui-backups"
RETENTION_DAYS=30
```

创建备份目录

```
mkdir -p $BACKUP_DIR
```

备份数据库

```
echo "备份数据库..."
mysqldump -u$DB_USER -p$DB_PASS --single-transaction --routines --triggers $DB_NAME | gz
```

备份文件

```
echo "备份文件..."
tar -czf "$BACKUP_DIR/files_$(date +%Y%m%d_%H%M%S).tar.gz" /var/www/xiaomotui/uploads
```

上传到云存储

```
echo "上传到云存储..."
```

```
aws s3 sync $BACKUP_DIR s3://$S3_BUCKET/$(date +%Y)/$(date +%m)/

# 清理旧备份

echo "清理旧备份..."
find $BACKUP_DIR -name "*.gz" -mtime +$RETENTION_DAYS -delete

echo "备份完成！"
```

7.4.2 恢复脚本

```
#!/bin/bash

# restore.sh - 数据恢复脚本

set -e

BACKUP_FILE=$1
DB_NAME="xiaomotui"
DB_USER="root"
DB_PASS=""

if [ -z "$BACKUP_FILE" ]; then
    echo "Usage: $0 "
    exit 1
fi

echo "恢复数据库..."
gunzip < $BACKUP_FILE | mysql -u$DB_USER -p$DB_PASS $DB_NAME

echo "恢复完成！"
```

附录

A. 错误码说明

错误码	说明	HTTP状态码
10001	参数错误	400
10002	用户不存在	404
10003	密码错误	401
10004	令牌无效	401
10005	权限不足	403
20001	内容生成失败	500
20002	设备离线	503
30001	支付失败	400
30002	余额不足	400

B. 配置参数说明

```
// 系统配置参数
return [
    // JWT配置
    'jwt' => [
        'secret_key' => env('JWT_SECRET'),
        'access_token_expire' => 7200, // 2小时
        'refresh_token_expire' => 604800, // 7天
    ],

    // 文件上传配置
    'upload' => [
        'max_size' => 10  1024  1024, // 10MB
        'allowed_ext' => ['jpg', 'png', 'gif', 'mp4', 'pdf'],
        'storage' => env('UPLOAD_STORAGE', 'local'), // local, qiniu, oss
    ],

    // 缓存配置
    'cache' => [
        'default_expire' => 3600, // 1小时
        'user_info_expire' => 1800, // 30分钟
        'config_expire' => 86400, // 24小时
    ],

    // 限流配置
    'rate_limit' => [
        'api' => [
            'requests' => 100,
            'window' => 60, // 1分钟
        ],
        'sms' => [
            'requests' => 5,
            'window' => 300, // 5分钟
        ],
    ],
];
```

C. 性能基准测试结果

指标	结果	达标情况
API响应时间(P95)	< 200ms	✓

API响应时间(P99)	< 500ms	✓
并发用户数	1000	✓
数据库查询时间	< 100ms	✓
缓存命中率	> 90%	✓
系统可用性	99.9%	✓

D. 常见问题与解决方案

Q1: 如何处理高并发下的库存超卖?
A: 使用Redis原子操作和消息队列，实现预减库存和延迟更新。

Q2: 大文件上传如何优化?
A: 采用分片上传、断点续传、异步处理等方案。

Q3: 如何保证数据一致性?
A: 使用分布式事务、最终一致性、幂等性设计等方法。

文档结束

本文档共30页，详细介绍了小磨推智能营销系统的技术架构、实现细节、性能优化、安全机制等核心内容，为系统的开发、部署、运维提供全面的技术指导。